

AN INTERNSHIP PROJECT REPORT

Automated Seller Performance Report Generation System

Submitted in partial fulfilment of the requirements of the Internship Programme

Prepared By	Himanshu Yadav
Role	Data Analyst Intern, Business Intelligence Team
Department	Business Intelligence Team
Organization	Shipway by Unicommerce
Domain	Logistics / Shipping Aggregation Analytics
Project Duration	June 2026 – August 2026
Mentor / Guide	Mayur Gautam
Submission Date	August 2026

Declaration

I hereby declare that the project titled “**Automated Seller Performance Report Generation System**”, submitted as part of my internship at Shipway by Unicommerce, is an original record of the work carried out by me under the guidance of my mentor. The work presented here has not been submitted elsewhere for the award of any other degree, diploma, or certificate.

Name **Himanshu Yadav**

Signature _____

Acknowledgement

I would like to express my sincere gratitude to my mentor and the entire Business Intelligence team at Shipway by Unicommerce for their continuous guidance, patience, and support throughout the course of this internship. Their feedback at every stage — from the first CSV-based prototype to the final Metabase-integrated, automated delivery pipeline — was instrumental in shaping this project into a production-usable system.

I am also thankful for the opportunity to work with real operational data and real production constraints, which taught me lessons about data engineering, API integration, and error handling that go well beyond what a classroom project typically covers.

Table of Contents

1. Introduction	
2. Project Overview	
3. Tools & Technologies Used	
4. System Architecture	
5. Detailed Working of the Script	
6. Pre-Processing Pipeline (Detailed)	
7. Usage Guide	
8. Testing & Validation	
9. Challenges Faced & Solutions	
10. Future Scope	
11. Conclusion	
12. Appendix	

1 Introduction

1.1 About the Organization

Shipway by Unicommerce is a logistics aggregator platform that connects e-commerce sellers with multiple courier partners (Delhivery, Ekart, Xpressbees, Shadowfax, Bluedart, and others), providing a single interface for order assignment, tracking, NDR (Non-Delivery Report) management, and shipment analytics. As an aggregator, Shipway's Business Intelligence (BI) function plays a central role: sellers and internal stakeholders (KAM, Customer Success, Operations) all depend on accurate, timely shipment performance data to make decisions.

1.2 About the Business Intelligence Team

The BI team is responsible for building and maintaining the reporting and analytics infrastructure that sits on top of Shipway's shipment database — courier SLA dashboards, seller performance scorecards, volume discount tracking, and ad-hoc reporting pipelines. Much of this work involves querying large shipment datasets (via Metabase, the company's BI/query platform), transforming raw operational data into business-ready metrics, and delivering it through Google Sheets dashboards, scheduled email reports, and self-serve request forms.

1.3 Objective of the Project

The objective of this project was to design and build an automated, self-serve system that lets internal stakeholders (KAMs, CS, Ops) request a Seller Performance Report for any Company ID and a chosen time window, without needing a BI analyst to manually run a query, clean the data, and email it every time. The system needed to:

- Accept requests through a simple Google Form (Company ID + reporting duration).
- Automatically pull the corresponding shipment data from Metabase.
- Apply the same business logic (TAT, SLA, NDR categorization, status grouping) used in manual reports.
- Deliver the result as a CSV, an Excel workbook, and a live Google Sheet — unattended, on a schedule.
- Do all of this reliably enough to run without a human watching every execution.

1.4 Scope

The scope covers the full pipeline: request intake, data extraction, data cleaning and business-rule computation, output generation in three formats (CSV, Google Sheet, XLSX), and email delivery with size-aware fallback to Google Drive. It does not cover the underlying Metabase

query itself (owned separately by the BI team) or the design of the downstream dashboard tabs the Google Sheet feeds — the project treats those as existing systems it integrates with, not systems it redesigns.

2 Project Overview

2.1 Problem Statement

Before this project, generating a seller performance report meant a BI analyst manually running a Metabase query for a specific company and date range, downloading the result, applying a fixed set of pandas transformations (TAT/SLA calculation, status normalization, etc.), and emailing the output by hand. This did not scale: every request consumed analyst time, turnaround depended on analyst availability, and the process was prone to inconsistency between requests (a slightly different filter here, a missed step there).

2.2 Proposed Solution

The solution reframes report generation as an automated pipeline triggered by a Google Form submission rather than an analyst's manual effort:

1. A requester fills a Google Form with a Company ID and a Duration choice.
2. The form response lands in a Google Sheet that acts as a request queue.
3. A scheduled Python script polls that queue for pending (unprocessed) rows.
4. For each pending row, it computes the correct date range, authenticates with Metabase, and pulls the shipment data for that company via a REST API call.
5. The raw data is cleaned, renamed to a stable business-facing schema, and enriched with computed KPI columns (TAT, SLA status, NDR category, etc.).
6. The result is saved as a CSV, pushed to a live Google Sheet (feeding existing dashboard tabs), and exported as a formatted Excel workbook.
7. The CSV (zipped) and the Excel workbook are emailed to the requester, with CC to the BI distribution list — falling back to Google Drive links if the files are too large to attach.
8. The request row is marked “Done” so it is never processed twice.

2.3 Why This Matters

This turns a multi-minute-to-multi-hour manual task (depending on analyst availability) into a self-serve request that resolves automatically, typically within a few minutes, at any volume — the same script handles a seller with a few hundred shipments and one with over 150,000, adjusting its behaviour (formatting strategy, attachment-vs-Drive-link delivery) based on data size rather than needing separate code paths maintained by hand.

3 Tools & Technologies Used

Category	Tool / Library	Purpose
Language	Python 3	Core scripting language for the entire pipeline
Data Handling	pandas	Data cleaning, transformation, KPI computation
HTTP Client	requests (with urllib3 Retry / HTTPAdapter)	Metabase API calls with automatic retry on transient failures
Data Source	Metabase REST API	Fetches shipment data via a parameterized saved question
Spreadsheet I/O	pygsheets	Reads the request-queue sheet; writes the Raw data tab
Spreadsheet Formatting	google-api-python-client (Sheets API v4)	Reads cell-level formatting (fills, borders, merges) for the Excel export
File Storage Fallback	Google Drive API	Uploads oversized reports and shares a link when email attachment limits are exceeded
Excel Generation	openpyxl	Builds the formatted .xlsx report without live formulas
Email Delivery	smtplib / email.message (Gmail SMTP)	Sends the report with attachments or Drive links, plus CC distribution
Date Logic	python-dateutil (relativedelta)	Computes calendar-aware date ranges for each Duration option
Request Intake	Google Forms	Self-serve UI for Company ID + Duration submission
Instant Acknowledgement	Google Apps Script (onFormSubmit trigger) + MailApp	Sends an immediate confirmation email the moment a form is submitted, ahead of the actual report
Compression	zipfile (standard library)	Compresses the CSV before emailing

4 System Architecture

4.1 High-Level Workflow

Two independent systems act on the same form submission: an instant Google Apps Script trigger that acknowledges the request within seconds, and a separately-scheduled Python pipeline that generates and delivers the actual report.



4.2 Component Description

Request Queue (Google Sheet) — A Google Form writes each submission as a new row into a linked Google Sheet, with columns for Timestamp, Email address, Company ID, Duration, and a Status column the script writes back to. Only rows with a blank Status are treated

as pending.

Instant Acknowledgement (Google Apps Script) — A container-bound Apps Script, triggered on the Sheet's onFormSubmit event, fires the moment a form is submitted — independently of, and well before, the Python pipeline actually processes the request. It reads the submitted row directly (timestamp, email, Company ID, Duration) and sends an immediate HTML confirmation email via MailApp, so the requester knows their submission was received without waiting for the report itself. The email also sets an expectation for when the report will actually arrive, based on a 1:00 PM cutoff: submissions before 1:00 PM are told to expect their report after 1:00 PM the same day; submissions after 1:00 PM are told to expect it after 7:00 PM. The same BI distribution list used on the final report email is CC'd here as well, so the team has visibility into every incoming request, not just the completed ones.

Metabase Integration — The script authenticates against Metabase via a session-token login, then calls the saved question's /query/json endpoint with three parameters bound to the question's own filter widgets: Start Date, End Date, and Company ID. Filtering happens inside Metabase's own query, not after downloading the full dataset — this matters because some sellers have well over 150,000 shipment rows in a single reporting window.

Transformation Layer — A single function, generate_report(), takes a Company ID and a Duration string and returns a finished CSV path. Internally it renames the database's raw column names to a stable, human-readable schema (so the underlying database schema can change without breaking downstream sheets), parses every date field defensively, computes all KPI columns, and assembles the final column layout in a fixed order.

Delivery Layer — Three delivery artifacts are produced: a CSV (zipped), a live Google Sheet update, and a formatted Excel workbook. The email step checks the combined attachment size against Gmail's practical limit and automatically switches to Google Drive links when the files are too large to attach directly.

5 Detailed Working of the Script

5.1 Request Intake

The script reads all rows from the request-queue sheet using pygsheets' `get_all_records()`, which returns each row as a dictionary keyed by column header. Rows are filtered down to those with a blank Status. For each pending row, the Company ID and Duration are validated before any API call is made: a blank Company ID or an unrecognized Duration value causes that row to be skipped (with a printed reason) rather than crash the whole batch or silently guess a default.

5.2 Data Extraction (Metabase Integration)

Authentication uses a username/password session login against Metabase's `/api/session` endpoint, returning a session token used on the subsequent data call. The data call itself hits `/api/card/{question_id}/query/json` with a JSON body containing the three parameters, matched to the exact variable names configured on the Metabase question's filter widgets. All HTTP calls go through a `requests.Session` configured with automatic retry (3 attempts, exponential backoff) on connection errors and 429/5xx responses, since this call sits on the critical path of every single run.

An explicit, generous timeout is set on every network call (Metabase session, Metabase query, the Sheets API client, and the SMTP connection) so that a stalled request fails within a known, bounded time with a clear error — rather than hanging indefinitely, which is what an unbounded default timeout allowed to happen in earlier iterations of this project.

5.3 Business Rule / KPI Computation

Once the raw data is loaded and renamed, the following derived columns are computed:

Column	Logic
Assign_Date / Attempt_Date	Date-only extraction from the Order Assign / First Attempt timestamps
speed	“Air” if the courier slab text contains “express”, else “Surface”
Month / Week_Month	Reporting-period labels derived from the Order Assign timestamp
O2P, O2D, P2A, P2D	Transit-time deltas in days between key milestones; negative values (out-of-order timestamps) are clamped to zero
TAT	Looked up from a speed × zone matrix (e.g. Surface + Rest of India = 7 days)
SLA_Status	“Within_TAT” or “TAT_Breached”, comparing the pickup-to-first-attempt time against TAT
Attempt_Category	Not_Attempt / FASR / NDR, based on total attempt count
Delivered_Category	FASR_Delivered vs NDR_Delivered, based on NDR attempt count at delivery
O2P_Compliance	Bucketed pickup speed: within 24h / 48h / more than 48h
Order_Status_Group	Normalized bucket (DELIVERED, RTO, IN TRANSIT, etc.) mapped from the raw status field

5.4 Output Generation

The final DataFrame is assembled in a fixed, legacy-compatible column order so that the destination Google Sheet's column positions never shift between runs — any column the current data source doesn't provide is still emitted, just left blank, and any new column the data source adds is appended at the end rather than inserted into the middle of the existing layout. The CSV is written locally and the Google Sheet's “Raw” tab is updated to match.

5.5 Excel Report Generation

The Excel workbook is not a native export of the Google Sheet — it is rebuilt cell-by-cell from the Sheets API's computed values and formatting, for two reasons: first, a native export keeps every live formula, and formulas built on Google-only functions (QUERY, ARRAYFORMULA) have no Excel equivalent and break with #NAME? errors the moment Excel tries to recalculate them; second, Excel cannot open a live connection back to the Google Sheet, so a static snapshot is the only correct deliverable in an email attachment. The rebuild reads each cell's formattedValue (the computed display text, never the formula) and effectiveFormat (background colour, bold, font colour, borders) plus the sheet's merged-cell ranges, and reproduces them in a fresh openpyxl workbook.

Large data tabs (the Raw tab and any other tab above a row-count threshold) are treated differently from small dashboard tabs: their per-cell formatting is not fetched at all, since a 150,000-row formatting request is what caused multi-minute stalls and timeouts in early versions. Instead they get a fast, values-only fetch with the header row bolded manually. The Raw tab itself is excluded from the Excel workbook entirely, since the same data is already delivered in full inside the zipped CSV.

5.6 Email Delivery

The email carries the zipped CSV and the Excel workbook as attachments, CC'd to a fixed BI distribution list. Before sending, the combined attachment size is checked against a safe margin under Gmail's 25MB hard limit; if exceeded, both files are instead uploaded to a Shared Drive folder and the email body contains view links rather than attachments. Once the email sends successfully, the corresponding request row's Status is updated to "Done" so it is never reprocessed.

5.7 Instant Acknowledgement Email (Google Apps Script)

The Python pipeline runs on a schedule, so there can be a real gap between a form submission and the moment its report is actually generated. Rather than leave the requester wondering whether their submission went through, a small Google Apps Script bound to the request Sheet responds within seconds of submission with a confirmation email. This is a genuinely separate system from the Python pipeline: it runs inside Google's own infrastructure, triggered by the Sheet's onFormSubmit event, and has no dependency on the Python script running, succeeding, or even being scheduled to run soon.

The acknowledgement email contains a short confirmation that the request was received; a small table restating the submitted Company ID, Duration, and exact request timestamp; an expected delivery window based on the 1:00 PM / 7:00 PM cutoff; and the same CC distribution list as the final report email, giving the BI team visibility into every incoming request as it arrives.

Design note: the 1:00 PM / 7:00 PM cutoff communicated here is a business commitment set independently of the Python script's actual run schedule. If the scheduled task's run times are ever changed, this cutoff logic needs to be updated to match — otherwise the acknowledgement email could promise a delivery window the pipeline isn't actually configured to meet.

6 Pre-Processing Pipeline (Detailed)

Pre-processing is the stage between “raw API response” and “business-ready DataFrame,” and is where most of this project's real engineering effort went, since the underlying data source's schema, date formats, and status vocabulary all needed to be normalized defensively rather than assumed stable.

6.1 Schema Mapping

The Metabase query returns columns in the database's own snake_case naming (company_id, order_time, courier_child, and so on). A single RENAME_MAP dictionary translates every one of these into the stable, human-readable schema (Company ID, Order Assign Date, Courier Slab, ...) that the rest of the pipeline and every downstream Google Sheet formula depends on. This indirection means the underlying database schema can be extended — as it was, several times, over the course of this project (Shipping State, Product Name, and Product Quantity columns were all added to the source mid-project) — without breaking anything downstream, since only the RENAME_MAP needs a one-line update.

6.2 Date Parsing Strategy

A smart_parse() function tries a cascade of date formats in order (ISO datetime, ISO date-only, legacy DD-MM-YYYY variants, and finally Excel serial-date numbers) so that a column can be parsed correctly whether it arrives as “2026-07-21 14:30:00” or as a raw numeric day-count, without the caller needing to know in advance which format a given data source will use.

6.3 EDD Fallback Handling

The Estimated Delivery Date is sourced from two possible upstream fields — shipway_edd and courier_edd. The pipeline uses shipway_edd whenever it is present and non-blank, falling back to courier_edd only when it is not, ensuring a single, reliable EDD Date column downstream regardless of which upstream system actually populated it for a given shipment.

6.4 Duration-Based Date Range Logic

The reporting window is entirely determined by the requester's Duration selection on the form, not by an automatic rule — an earlier version of this project computed the window automatically based on how far into the current month it was, but this was replaced with explicit requester choice for predictability. Five options are supported:

Duration Option	Range Start	Range End
Last 3 Months	1st of the month, 3 months before the current month	Last day of the previous month
Current Month and Last 2 Months	1st of the month, 2 months before the current month	Today
Current Month and Last 1 Month	1st of the month, 1 month before the current month	Today
Current Month	1st of the current month	Today
Last Month	1st of the previous month	Last day of the previous month

Any Duration value that does not exactly match one of these five (a typo, a form edited later without a matching code update) causes the request to fail loudly with a clear error, rather than silently falling back to a default and shipping a report for the wrong window.

6.5 Status Taxonomy Normalization

The raw status field mixes two vocabularies: human-readable statuses with underscores (RTO_DELIVERED, PICKUP_EXCEPTION) and raw internal shipment-status codes (SHPFR#, SHNDR#, RTONDR#, and similar). A mapping table handles the known human-readable values directly, while a prefix-matching fallback groups the coded values by family (e.g. every SHNDR-prefixed code is treated as an undelivered-attempt status). Each prefix's target bucket was verified, not assumed — cross-checked against a second, independently computed status field already present in the source data, with any disagreement between the two logged as a QA count for manual review rather than silently trusted.

6.6 Safety-Net Filtering

Two defensive filters run after the main transformation, even though the Metabase query itself already filters by Company ID and date range: a client-side Company ID filter (in case a query parameter is ever not wired into every part of the underlying SQL) and a client-side date-range filter using the exact same computed window. Neither should ever remove rows in normal operation — their purpose is to fail safe if the upstream filtering is ever incomplete, rather than silently ship a report that includes data it shouldn't.

7 Usage Guide

7.1 Prerequisites

- Python 3.9+ with pandas, requests, pygsheets, openpyxl, google-api-python-client, google-auth-httplib2, python-dateutil installed.
- A Google Cloud service account with access to: the request-queue Sheet, the output Sheet, and a Shared Drive folder (for oversized report fallback).
- A Metabase account with access to the saved question used for data extraction, and that question's numeric ID.
- A Gmail account (or app-password-enabled SMTP account) for sending report emails.

7.2 Configuration

All configuration lives in a single CONFIG block at the top of the script:

```
SERVICE_ACCOUNT_FILE = "client_secret.json"
REQUESTS_SHEET_KEY    = "<Google Sheet ID of the request queue>"
REQUESTS_TAB          = "Form responses 1"
OUTPUT_SHEET_KEY      = "<Google Sheet ID of the output/dashboard sheet>"
EMAIL / EMAIL_PASSWORD = "<sender Gmail address / app password>"
CC_EMAILS             = [ ... BI distribution list ... ]
METABASE_URL          = "https://metabase.<company>.com"
USERNAME / PASSWORD   = "<Metabase login>"
QUESTION_ID           = <numeric Metabase question ID>
DRIVE_UPLOAD_FOLDER_ID = "<Shared Drive folder ID, for oversized reports>"
```

7.3 The Google Form

The request form has two required fields: **Company ID** — free-text, matched exactly against the Metabase parameter — and **Duration** — a multiple-choice field whose options must exactly match the five strings the script recognizes (see Section 6.4). Form submissions land automatically in the linked “Form responses 1” tab, which the script polls for unprocessed rows.

7.4 Running the Script

The script is designed to run unattended on a schedule (e.g. Windows Task Scheduler):

```
python Performance_Report_Automation.py
```

It requires no command-line arguments — each run polls the request queue and processes every pending row it finds in that pass. Running it more frequently simply reduces the average wait time between a form submission and its report being delivered.

7.5 Expected Output

For each processed request, three artifacts are produced:

- A CSV file: {Company ID}_{Company Name}_{date}.csv, zipped for delivery.
- An updated “Raw” tab in the output Google Sheet, feeding the existing dashboard tabs.
- An Excel workbook: {same name}_Performance.xlsx, containing every dashboard tab with formatting preserved, excluding the raw data tab.

The requester (and the CC list) receives an email with these attachments, or Drive links if the files exceed the size limit for direct attachment.

8 Testing & Validation

Because this pipeline touches several external systems (Metabase, Google Sheets, Google Drive, SMTP), most components were tested using mocked versions of those systems rather than live calls, so that logic could be verified quickly and repeatably without depending on network access or real credentials during development.

- Date-range logic was tested against all five Duration options plus an invalid option, confirming exact start/end dates and confirming the invalid case fails loudly.
- The full transformation pipeline was run end-to-end against real, previously-exported shipment data to confirm row counts, computed KPI columns, and the status-mapping QA cross-check all behaved as expected.
- The Excel export was tested against a mocked Sheets API response containing a merged, bordered title cell and a bold header row, confirming both survived the rebuild.
- The email size-check logic was tested with both a small (attach) and an artificially large (Drive-link fallback) case, confirming attachments are correctly included or omitted.
- The Google Sheets resize logic was tested to confirm the Raw tab only ever grows or is left unchanged — never shrunk — after a formula-breaking #REF! issue was traced back to an earlier shrink-then-grow approach.

The built-in QA cross-check (Order_Status_Group vs. the source's own Shipment_Status_Raw bucket) also acts as an ongoing validation mechanism in production: every run prints a count of disagreeing rows, giving an early signal if the underlying status vocabulary changes in a way the mapping table hasn't accounted for yet.

9 Challenges Faced & Solutions

This project evolved iteratively from a manual, file-based script into a fully automated pipeline, and each stage of that evolution surfaced a real production problem that needed a real fix — not just a workaround.

Challenge	Root Cause	Resolution
SSL certificate verification failure connecting to Metabase	Internally-issued certificate missing a required chain attribute	Added a configurable SSL-verification toggle, with a documented security trade-off, plus guidance to switch to a proper internal CA bundle when available
Metabase query returning unusable data via one API endpoint	Parameters were being sent to the wrong endpoint/format (form-encoded CSV export vs. JSON body)	Switched to the /query/json endpoint with parameters sent as a genuine JSON body, matching a proven working reference pattern
Cryptic pandas error (“.str accessor... not integer”) on some companies	Metabase returning zero rows for a company/date combination produces a DataFrame with no columns at all	Added an explicit empty-result check immediately after the fetch, with a clear, actionable error message
Script hanging and eventually timing out on large sellers	Fetching full per-cell formatting for a 50,000+ row raw-data tab is an enormous, slow request	Split the export logic: large tabs get a fast values-only fetch; only small dashboard tabs get full formatting
Email delivery failing for very large sellers	Combined attachment size exceeding Gmail's 25MB limit	Added a size check that automatically switches to Google Drive upload + shareable links above a safe threshold
“Above the limit of 10,000,000 cells” error from the Sheets API	Google Sheets enforces a hard cell-count ceiling across the whole spreadsheet, and tabs left oversized from earlier runs quietly ate into that budget	Made the Raw tab strictly grow-only, and identified that auto-shrinking other tabs was unsafe once it was confirmed they all carry live formulas
#REF! errors appearing on dashboard tabs after a run	Shrinking a tab's grid dimensions permanently breaks any formula elsewhere referencing a range beyond the new, smaller size — growing it back does not undo this	Removed all automatic shrinking of any tab; the Raw tab now only ever grows or is left unchanged

10 Future Scope

- Move the Shared Drive-based large-file fallback to also cover the live Google Sheet update, for sellers so large the Sheet itself becomes impractical to browse.
- Add a lightweight monitoring/alerting layer (e.g. a Slack notification) for any request that fails processing, instead of relying on someone reading the script's console output.
- Allow the CC list to be configured per-request from the form itself, rather than as a single fixed distribution list for every report.
- Migrate large, rarely-queried historical data tabs out of the shared output spreadsheet into a separate workbook, permanently resolving the 10,000,000-cell budget contention rather than just working around it.
- Add automated regression tests that run against a small, checked-in sample dataset on every code change, rather than only ad-hoc manual verification.

11 Conclusion

This project started as a straightforward CSV-processing script and, through several rounds of real production feedback, grew into a fully automated, self-serve reporting pipeline integrating a request form, a live BI query platform, a shared dashboard spreadsheet, and email delivery with size-aware fallback logic. Along the way it surfaced and required fixing genuine platform-level issues — API authentication quirks, spreadsheet cell-limit constraints, formula-breaking resize behaviour, and email attachment limits — that are not typically covered in a classroom setting but are exactly the kind of problem real production data systems run into.

The resulting system is in active use by the BI team, reduces analyst time spent on repetitive manual report requests, and is built to scale to Shipway's largest sellers without needing separate code paths for “small” and “large” cases — the same script adapts its own formatting and delivery strategy based on the data it encounters at runtime.

12 Appendix

12.1 Full Output Column List

The final CSV / Google Sheet output contains the following columns, in this fixed order:

#	Column Group	Representative Columns
1	Identifiers	Company ID, Company Name, Order ID, Vendor Order ID, Tracking Number
2	Courier Info	Courier ID, Parent Courier, Courier Slab, speed
3	Status	Status, Sub Status, Order_Status_Group, Shipment_Status_Raw, Delivered_Category, Attempt_Category
4	Dates	Order Assign Date, Pickup Date, First Attempt Date, Delivery Date, EDD Date, RTO Initiated Date, RTD Date
5	NDR Details	NDR Last NDR Status, NDR Last/Second/Third Attempt Date, NDR Attempt Count, Total Attempt Counts
6	Commercial	Total Order Amount, Payment Method
7	Geography	Shipping/Pickup Pincode, Zone, Zone_clean, Shipping State, Shipping/Pickup City
8	Product	Product Name, Product Quantity
9	Computed KPIs	O2P, O2D, P2A, P2D, TAT, SLA_Status, O2P_Compliance, Month, Week_Month
10	Legacy placeholders	AWB Shipped Date, Follow-up Mode, Last Customer response, NDR/RTO Scan Reason (kept blank if unavailable from the source)

12.2 Key Configuration Constants

Constant	Purpose
VALID_DURATIONS	The set of five accepted Duration strings from the form
GMAIL_SAFE_ATTACHMENT_MB	Threshold (20MB) below which files are attached directly rather than uploaded to Drive
RENAME_MAP	Maps the data source's raw column names to the stable output schema
STATUS_GROUP_MAP	Maps raw status values/codes to normalized Order_Status_Group buckets
TAT_MAP	Speed × zone lookup table for expected turnaround time in days
FINAL_COL_ORDER	The fixed column order for every CSV / Sheet output

12.3 Google Apps Script — Instant Acknowledgement Email

Bound to the request Sheet, configured with an installable trigger: Event Source “From Spreadsheet,” Event Type “On Form Submit.” Fires within seconds of every form submission, independently of the Python pipeline described in Section 5.7.

```
/**
 * Trigger:
 * Event Source : From Spreadsheet
 * Event Type   : On Form Submit
 */
function onFormSubmit(e) {
  // Get submitted row
  var row = e.range.getRow();
  var sheet = e.range.getSheet();

  // Read values
  var sc-camel-submission-time = new Date(sheet.getRange(row, 1).getValue());
  var email = sheet.getRange(row, 2).getValue();
  var company = sheet.getRange(row, 3).getValue();
  var duration = sheet.getRange(row, 4).getValue();

  // 1 PM cutoff
  var cutoff = new Date(submissionTime);
  cutoff.setHours(13, 0, 0, 0);

  var reportTime;
  if (submissionTime < cutoff) {
    sc-camel-report-time = "today after <b>1:00 PM</b>";
  } else {
    sc-camel-report-time = "today after <b>7:00 PM</b>";
  }

  // CC Recipients
  var sc-camel-cc-emails = [
    "reports@shipway.com"
  ].join(",");

  // Email Body
  var sc-camel-html-body =
    "<p>Dear User,</p>" +
    "<p>Thank you for submitting your <b>Seller Performance Report</b> request.</p>" +
    "<p>Your request has been received successfully.</p>" +
    "<p><b>Request Details</b></p>" +
    "<table border='1' cellpadding='8' cellspacing='0' " +
    "style='border-collapse:collapse;font-family:Arial;font-size:13px;'>"
    + "<tr>"
    + "<td><b>Company ID</b></td>"
    + "<td>" + company + "</td>"
    + "</tr>"

```

```

+ "<tr>"
+ "<td><b>Duration</b></td>"
+ "<td>" + duration + "</td>"
+ "</tr>"
+ "<tr>"
+ "<td><b>Request Time</b></td>"
+ "<td>" +
    Utilities.formatDate(
        submissionTime,
        Session.getScriptTimeZone(),
        "dd-MMM-yyyy hh:mm a"
    )
+ "</td>"
+ "</tr>"
+ "</table>"
+ "<br>"
+ "<p>Your report will be shared with you <b>" +
    reportTime +
    "</b>.</p>"
+ "<br>"
+ "<p>If you have any questions, please feel free to contact the BI Team.</p>"
+ "<br><br>"
+ "<p>Regards,<br>"
+ "<b>BI Team</b></p>";

// Send Email
MailApp.sendEmail({
    to: email,
    cc: ccEmails,
    subject: "Seller Performance Report Request Received | Company ID: " + company,
    htmlBody: htmlBody
});
}

```